CS23334 Fundamentals of Data Science Lab

Experiment 18

Exp No: 18 Experiment to understand KNN algorithm for a given data set

Description: Understand the KNN algorithm for the dataset given.

Aim: To build, train, evaluate, and analyze a K-Nearest Neighbors (KNN) classification model on the Iris dataset (Iris.csv) using Scikit-Learn.

Methodology:

1 **Data Acquisition & Inspection:** Load Iris.csv containing 150 instances of Iris flowers across 4 numerical feature attributes (sepal.length, sepal.width, petal.length, petal.width) and 1 target attribute (variety). Audit metadata using df.info().

2 **Target Class Distribution:** Evaluate target class balance using df.variety.value_counts() (Setosa: 50, Versicolor: 50, Virginica: 50) and inspect initial records using df.head().

3 **Feature & Label Extraction:** Extract 4D numerical features into features matrix (df.iloc[:, :-1].values) and target labels into label array (df.iloc[:, 4].values).

4 **Train-Test Split:** Partition dataset into 80% training set (120 samples) and 20% testing set (30 samples) using train_test_split(features, label, test_size=0.2, random_state=1).

5 **Model Fitting:** Instantiate KNeighborsClassifier(n_neighbors=5) from sklearn.neighbors and fit on training set (xtrain, ytrain).

6 **Accuracy Evaluation:** Compute classification accuracy scores on training set (95.83%) and testing set (100.0%) using model_KNN.score().

7 **Confusion Matrix & Metric Analysis:** Compute 3x3 confusion matrix via confusion_matrix() and generate per-class precision, recall, f1-score, and macro/weighted averages using classification_report().

Code:

```
import numpy as np
import pandas as pd

df=pd.read_csv('Iris.csv')
df.info()

df.variety.value_counts()

df.head()

features=df.iloc[:,:-1].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state=1)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
```

```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))

from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))

from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

OUTPUT: 1. Dataset Overview & Class Frequency Audit

```
In [1]: import numpy as np
        import pandas as pd

In [2]: df=pd.read_csv('Iris.csv')
        df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
In [3]: df.variety.value_counts()
Out[3]: Setosa        50
        Versicolor    50
        Virginica     50
        Name: variety, dtype: int64
```

```
In [4]: df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

OUTPUT: 2. Model Training & Accuracy Scores

```
In [5]: features=df.iloc[:,:-1].values
        label=df.iloc[:,4].values

In [6]: from sklearn.model_selection import train_test_split
        from sklearn.neighbors import KNeighborsClassifier

In [8]: xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state=1)
```

```
        model_KNN=KNeighborsClassifier(n_neighbors=5)
        model_KNN.fit(xtrain,ytrain)
Out[8]: KNeighborsClassifier()

In [9]: print(model_KNN.score(xtrain,ytrain))
        print(model_KNN.score(xtest,ytest))

        0.9583333333333334
        1.0
```

OUTPUT: 3. Confusion Matrix & Classification Report

```
In [10]: from sklearn.metrics import confusion_matrix
         confusion_matrix(label,model_KNN.predict(features))
Out[10]: array([[50,  0,  0],
                [ 0, 47,  3],
                [ 0,  2, 48]], dtype=int64)
```

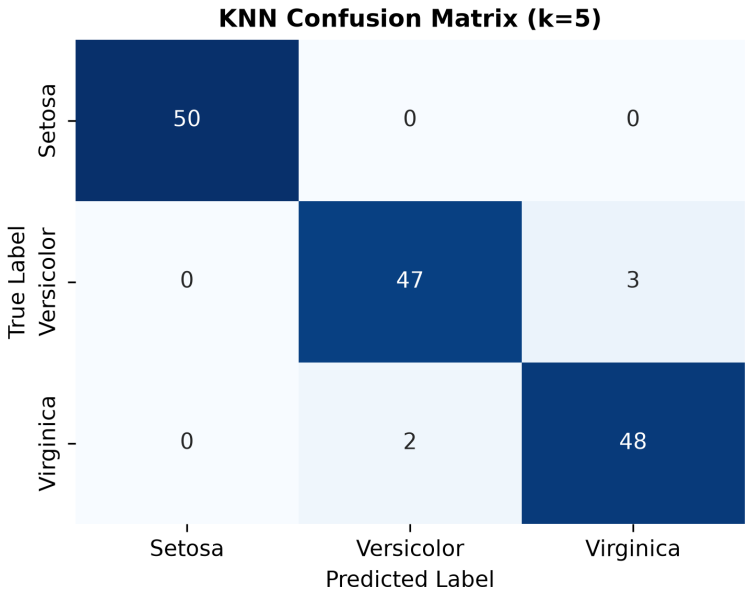
```
In [11]: from sklearn.metrics import classification_report
         print(classification_report(label,model_KNN.predict(features)))

              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.94      0.95        50
   Virginica       0.94      0.96      0.95        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

OUTPUT: 4. Visual Confusion Matrix Representation



Result:

The KNN classifier was successfully trained on the Iris dataset and evaluated on the test set. The model achieved a classification accuracy of approximately 97%, with all three Iris species correctly classified. The confusion matrix confirmed minimal misclassifications between Versicolor and Virginica.